Exercise 3

Joshua John E. Santiago

2026-10-15

Part A — Creating Vectors: c(), seq(), rep() # TODO: Create a numeric vector called scores containing: 88, 92, 79, 95, 84 # Then create a character vector called subjects containing: # “Math”, “Science”, “English”, “History”, “PE”

```
# Numeric vector of scores
scores <- c(88, 92, 79, 95, 84)

# Character vector of subjects
subjects <- c("Math", "Science", "English", "History", "PE")
```

TODO: Use seq() to create a vector called odds containing odd numbers

from 1 to 19 (hint: use the “by” argument)

TODO: Use seq() with length.out to create a vector of exactly 5 evenly

spaced numbers between 0 and 1

```
# Odd numbers from 1 to 19
odds <- seq(1, 19, by = 2)

# 5 evenly spaced numbers between 0 and 1
even_spaced <- seq(0, 1, length.out = 5)
```

TODO: Use rep() to create a vector that repeats the value “A” 4 times

TODO: Use rep() with the “each” argument to turn c(“A”, “B”) into

“A” “A” “B” “B” (each element repeated twice, in order)

TODO: Use rep() with the “times” argument to turn c(“A”, “B”) into

“A” “B” “A” “B” (the whole sequence repeated twice)

```
# "A" repeated 4 times
rep("A", times = 4)
```

```
## [1] "A" "A" "A" "A"
```

```
# Each element repeated twice, in order
rep(c("A", "B"), each = 2)
```

```
## [1] "A" "A" "B" "B"
```

```
# Whole sequence repeated twice
rep(c("A", "B"), times = 2)
```

```
## [1] "A" "B" "A" "B"
```

Question #1: Set up a vector named age, consisting of 15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 37, 46, 25, 17, 12, 42, 53, 41, 51, 35, 24, 33, 8.

```
age <- c(15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 37, 46, 25
```

a.How many data points?

```
age <- c(15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 37, 46, 25

length(age)
```

```
## [1] 34
```

#answer: 34 data points

b.Find the reciprocal of age? #answer:

```
age <- c(15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 37, 46, 25
1 / age
```

```
##  [1] 0.06666667 0.03571429 0.03125000 0.02777778 0.03703704 0.05555556
##  [7] 0.01923077 0.02564103 0.02380952 0.03448276 0.02857143 0.03225806
## [13] 0.03703704 0.04545455 0.02702703 0.02941176 0.05263158 0.05000000
## [19] 0.01754386 0.02040816 0.02000000 0.02702703 0.02173913 0.04000000
## [25] 0.05882353 0.08333333 0.02380952 0.01886792 0.02439024 0.01960784
## [31] 0.02857143 0.04166667 0.03030303 0.12500000
```

c.Assign also new_age <- c(age, 0, age). What happens to new_age? #answer:

```r
new_age <- c(age, 0, age)
length(new_age)
```

```
## [1] 69
```

d: Find the minimum and the maximum value of age

```r
min(age)
```

```
## [1] 8
```

```r
max(age)
```

```
## [1] 57
```

#answer: minimum age is 8 and the maximum age is 57.

Question #2:

rep(c("A","B"), each = 2) and rep(c("A","B"), times = 2) both produce a vector of length 4, but with different element orders. Explain in your own words the difference between each and times. #answer: each repeats each element right away before moving to the next one, so you get "A" "A" "B" "B". times repeats the whole vector again, so you get "A" "B" "A" "B".

Question #3: Compare seq(1, 20, by = 2) with seq(1, 20, length.out = 5). What does each argument (by vs length.out) control, and when would you use one over the other in a real analysis? #answer: by sets the gap between numbers, so seq(1, 20, by = 2) gives 1 3 5 7 ... 19. length.out sets how many numbers you get, so seq(1, 20, length.out = 5) gives 1.00 5.75 10.50 15.25 20.00.I'd use by when I know the step I want, like every 2 units. I'd use length.out when I need a certain number of points, like 5 evenly spaced values for a graph.

Question #4: a.Generate a sequence for 20 to 60. Name the vector as vec_num

```r
vec_num <- 20:60
vec_num
```

```
##  [1] 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44
## [26] 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
```

b.Get the mean for vec_num.

```r
mean(vec_num)
```

```
## [1] 40
```

c.Get the sum for vec_num.

```r
sum(vec_num)
```

```
## [1] 1640
```

d.How many data points are there in vec_num?

```r
length(vec_num)
```

```
## [1] 41
```

Part B — Subsetting (Accessing) Vectors #TODO: Access the 3rd element of grades using positional indexing

```r
grades <- c(88, 92, 79, 95, 84)
names(grades) <- c("Math", "Science", "English", "History", "PE")

grades[3]
```

```
## English
##      79
```

#TODO: Access the 1st and 4th element of grades using positional indexing

```
grades[c(1, 4)]
```

```
##    Math History
##      88      95
```

#TODO: Access “Science” and “PE” together using their names

```
grades[c("Science", "PE")]
```

```
## Science      PE
##      92      84
```

#TODO: Return all elements of grades EXCEPT the 1st and 4th, using # negative indexing, e.g. grades[-c(...)]

```
grades[-c(1, 4)]
```

```
## Science English      PE
##      92      79      84
```

#TODO: Use a logical condition to return only the grades that are # greater than or equal to 85 (hint: grades[grades >= 85])

```
grades[grades >= 85]
```

```
##    Math Science History
##      88      92      95
```

Question #5: Negative indexing (grades[-1]) removes an element, while logical indexing (grades[grades >= 85]) filters by a condition.

Describe a real situation where you would need negative indexing, and a different situation where logical indexing would be the better tool. #answer: I’d use negative indexing when I just want to leave out one spot, like skipping the first value because it’s a header or a test entry: grades[-1]. It doesn’t actually delete anything, it just gives me a copy without that item. I’d use logical indexing when I want to filter by a rule, like finding all the subjects where my score is 85 or higher: grades[grades >= 85]. It’s better there because I don’t have to count positions, R checks every value for me.

Question #6:

What happens if you run grades[10] (an index that doesn’t exist)? Run it, observe the result, and explain why R responds this way instead of throwing an error.

```
grades[10]
```

```
## <NA>
##   NA
```

#answer:It gives `NA` instead of an error. Since `grades` only has 5 elements, position 10 doesn’t exist, so R just says there’s no value there. I think R does this so a whole script doesn’t crash over one missing spot, and you can spot the `NA` later and deal with it.

Question #7: Print a vector with the integers between 1 and 100 that are not divisible by 3, 5 and 7 using filter option. Filter(function(i) { all(i %% c(3,5,7) != 0) }, seq(100)) Write the R code and its output.

```
Filter(function(i) { all(i %% c(3, 5, 7) != 0) }, seq(100))
```

```
##  [1]  1  2  4  8 11 13 16 17 19 22 23 26 29 31 32 34 37 38 41 43 44 46 47 52 53
```

```
## [26] 58 59 61 62 64 67 68 71 73 74 76 79 82 83 86 88 89 92 94 97
```

Part C — Vector Operations and append() # TODO: Add x and y element-wise, and print the result

```
x <- c(1, 2, 3, 4)
y <- c(10, 20, 30, 40)

x + y
```

```
## [1] 11 22 33 44
```

TODO: Multiply every element of x by 2

```
x * 2
```

```
## [1] 2 4 6 8
```

TODO: Add z and short together and print the result. Run this even

though the vectors are different lengths – observe what R does

```
z <- c(1, 2, 3, 4, 5, 6)
short <- c(1, 0)

z + short
```

```
## [1] 2 2 4 4 6 6
```

TODO: Use append() to add “date” to the end of fruits

TODO: Use append() with the “after” argument to insert “kiwi” between

“apple” and “banana” (i.e., after position 1)

```
fruits <- c("apple", "banana", "cherry")

fruits <- append(fruits, "date")
fruits
```

```
## [1] "apple"  "banana" "cherry" "date"
```

```
fruits <- append(fruits, "kiwi", after = 1)
fruits
```

```
## [1] "apple"  "kiwi"   "banana" "cherry" "date"
```

Question #8: In the vectors-recycling chunk, z has 6 elements and short has only 2. R still produced an answer instead of an error. Explain, in your own words, what R’s “recycling rule” did to short to make the addition work #answer:R’s recycling rule repeats the shorter vector from the start until it’s as long as the longer one. So short (1, 0) was stretched into 1, 0, 1, 0, 1, 0, and then each value was added to the matching spot in z. It worked with no warning because 6 is a multiple of 2, so short fit in exactly three times.

Question #9: Now try adding z (length 6) to c(1, 0, 1) (length 3) instead. Does it work cleanly?

```
z + c(1, 0, 1)
```

```
## [1] 2 2 4 5 5 7
```

#answer:Yes, it works cleanly with no warning. R repeats c(1, 0, 1) twice to make 1, 0, 1, 1, 0, 1, which matches the length of z, because 3 fits into 6 exactly.

Then try adding z to c(1, 0, 1, 1) (length 4). Compare the warning (or lack of one) in each case, and explain what condition determines whether R recycles silently versus with a warning

```
z + c(1, 0, 1, 1)
```

```
## Warning in z + c(1, 0, 1, 1): longer object length is not a multiple of shorter
## object length
```

```
## [1] 2 2 4 5 6 6
```

#answer:This time R still gives an answer, but with a warning. It repeats c(1, 0, 1, 1) once, then only uses the first 2 values (1, 0) to fill the last two spots, so the pattern gets cut off in the middle.Comparing the two: length 3 gave no warning because it fit into 6 exactly twice, while length 4 gave a warning because it doesn't fit evenly. R recycles silently when the longer length is a multiple of the shorter length, and it warns when it isn't.

Part D — Lists: Creating and Subsetting # TODO: Create a list called student with these named elements: # name = "Maria Santos", age = 20, grades = c(88, 92, 79), # is_enrolled = TRUE

```
student <- list(name = "Maria Santos", age = 20,
grades = c(88, 92, 79), is_enrolled = TRUE)
student
```

```
## $name
## [1] "Maria Santos"
##
## $age
## [1] 20
##
## $grades
## [1] 88 92 79
##
## $is_enrolled
## [1] TRUE
```

TODO: Access the "grades" element of student using single brackets student["grades"] – then check its type with class()

```
student["grades"]
```

```
## $grades
## [1] 88 92 79
```

```
class(student["grades"])
```

```
## [1] "list"
```

TODO: Access the "grades" element of student using double brackets student[["grades"]] – then check its type with class()

```
student[["grades"]]
```

```
## [1] 88 92 79
```

```
class(student[["grades"]])
```

```
## [1] "numeric"
```

TODO: Access the "name" element of student using the $ operator student$name

```
student$name
```

```
## [1] "Maria Santos"
```

TODO: From student, extract just the SECOND grade in the grades vector

in a single expression, e.g. student[["grades"]][2] or student$grades[2]

```
student[["grades"]][2]
```

```
## [1] 92
```

Question #10: student["grades"] and student[["grades"]] look similar but return objects of different classes.

Run class() on each and explain the difference in what single-bracket [ ] versus double-bracket [[ ]] return from a list.

```
class(student["grades"])
```

```
## [1] "list"
```

```
class(student[["grades"]])
```

```
## [1] "numeric"
```

Question #11: Why do you think R makes a distinction between [ ] and [[ ]] for lists, but a vector only needs [ ]? Think about what a list can contain that a vector cannot. #answer:A vector can only hold one type of value, and each element is just a single value, so [ ] is enough to get what you want. A list can hold different things at once, like text, numbers, and even whole vectors or other lists. So R needs two options: [ ] to get a smaller list (the box), and [[ ]] to get the actual item inside it (what's in the box).

Part E — Critical Thinking Challenge

TODO: Using what you learned in Parts B and D, print only the items

(from inventory$item) that are currently in stock (in_stock == TRUE)

```
inventory <- list(
  item = c("Notebook", "Pen", "Eraser"),
  price = c(25.50, 10.00, 5.00),
  in_stock = c(TRUE, TRUE, FALSE)
)

inventory$item[inventory$in_stock]
```

```
## [1] "Notebook" "Pen"
```

TODO: A new product "Ruler" priced at 15.00 and in stock (TRUE) needs

to be added to inventory. Use append() to add "Ruler" to

inventory$item, 15.00 to inventory$price, and TRUE to inventory$in_stock.

(You will need three separate append() calls, one per vector.)

```
inventory$item     <- append(inventory$item, "Ruler")
inventory$price    <- append(inventory$price, 15.00)
inventory$in_stock <- append(inventory$in_stock, TRUE)

inventory
```

```
## $item
## [1] "Notebook" "Pen"      "Eraser"   "Ruler"
##
## $price
## [1] 25.5 10.0  5.0 15.0
##
## $in_stock
## [1]  TRUE  TRUE FALSE  TRUE
```

Question #12: In challenge-1, you combined logical indexing on one vector (in_stock) to filter a different vector (item) inside the same list. Explain, step by step, why inventory$item[inventory$in_stock] works even though item and in_stock are stored as separate elements of the list. #answer:First the inventory$in_stock pulls out the logical vector TRUE TRUE FALSE. Second inventory$item pulls out the character vector "Notebook" "Pen" "Eraser".Third Putting the logical vector inside [ ] tells R to check each position: keep the item if it's TRUE, drop it if it's FALSE.Fourth It works because both vectors have the same length and the same order, so position 1 in in_stock matches position 1 in item, and so on. R doesn't care that they're stored separately in the list, it only matches by position.So Notebook and Pen are kept, and Eraser is dropped.

Question #13: Suppose a classmate stores related data as three separate, unconnected vectors (item_names,

item_prices, item_stock) instead of grouping them into one list like inventory. What risks or bugs could this create as the dataset grows or gets reordered? Argue for or against using a list (or, later, a data frame) instead of separate vectors. #answer:Separate vectors can easily go out of sync. If I sort or delete from item_names but not item_prices, the prices end up on the wrong items and R gives no error. This gets harder to catch as the data grows.I'd use a list or data frame, since it keeps item, price, and stock together so they stay matched.

Conclusion In 3–4 sentences, summarize the key differences between vectors and lists in R, and explain when you would choose one data structure over the other. #answer:A vector holds only one type of data, while a list can hold different types together, like text, numbers, and even other vectors. Vectors use [ ] for subsetting, while lists use [ ] to get a smaller list and [[ ]] or $ to get the item inside. I'd use a vector for simple data of one kind, like a set of grades. I'd use a list when I need to group different details that belong together, like a student's name, age, and grades.